

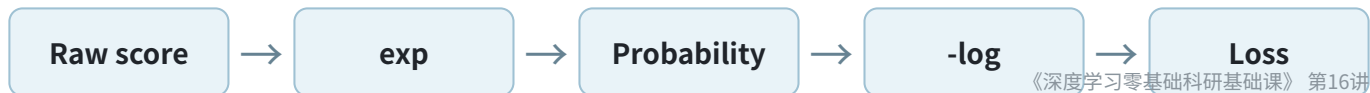
第16讲：深度学习需要的最小概率、指数与对数

Just-in-Time Math：只学 Softmax、Cross Entropy 与 LLM 真正需要的概率数学

本讲核心问题：为什么分类模型最终要输出“概率”？为什么 Softmax 里要用 $\exp$ ？为什么 Cross Entropy 里会出现 $\log$ ？

学完以后你应该能做到

- 理解概率为什么必须位于 0 到 1，并且一个离散分布的概率总和为 1。
- 理解指数函数 $\exp(x)=e^x$ 为什么始终为正，并会放大分数差异。
- 理解自然对数 $\ln(x)$ 是 $\exp$ 的逆运算，以及“乘法变加法”的关键性质。
- 理解 $-\log(p)$ 为什么能把“正确答案概率低”变成“大惩罚”。
- 能够把这些数学重新连接到 Softmax、Cross Entropy 和 LLM next-token prediction。



1. 为什么深度学习分类最终需要“概率”

回归任务可以直接预测一个连续数值，例如温度 23.7 或延迟 120ms；但分类任务面对的是“多个候选答案选哪个”。此时我们通常希望模型不仅说“选 A”，还告诉我们它对 A、B、C 分别有多大把握。

例子：三个候选 Token 的概率可能是 [0.70, 0.20, 0.10]。它表达的不只是“第一个最大”，还表达模型对三个候选的相对信心。

一个离散概率分布至少满足：

$$0 \leq p_i \leq 1 \quad \text{且} \quad \sum p_i = 1$$

向量	是否合法	原因
[0.2, 0.3, 0.5]	是	每项在 [0,1]，总和为 1
[0.2, 0.3, 0.4]	否	总和只有 0.9
[-0.1, 0.4, 0.7]	否	出现负概率

LLM 连接：对每个位置，模型最终要在整个 vocabulary 上形成一个概率分布。若词表大小 $V=50,000$ ，那么每个 Token 位置最终会得到 50,000 个候选概率。

2. Probability、Score、Logit：不要混成一个概念

神经网络最后一层常常先输出一组没有概率约束的实数，这些数通常叫 **scores** 或 **logits**。

logits = [2.0, 1.0, -0.5]

它们可以是正数、负数，也不要求总和为 1，因此不能直接解释成概率。

概念	允许范围	总和必须为1吗	用途
Logit / raw score	任意实数	否	模型原始偏好分数
Probability	[0,1]	离散分布通常是	表达相对可能性

常见误区：“最大的 logit 就是概率最高的类别”在排序意义上通常成立，但 logit 的数值本身不是概率。例如 logit=8 并不表示 80% 或 8%。

接下来的数学问题就是：怎样把任意实数 logits 变成正数，并进一步变成合法概率？这就是指数函数开始出现的地方。

3. 指数函数 $\exp(x)=e^x$ ：它到底做了什么

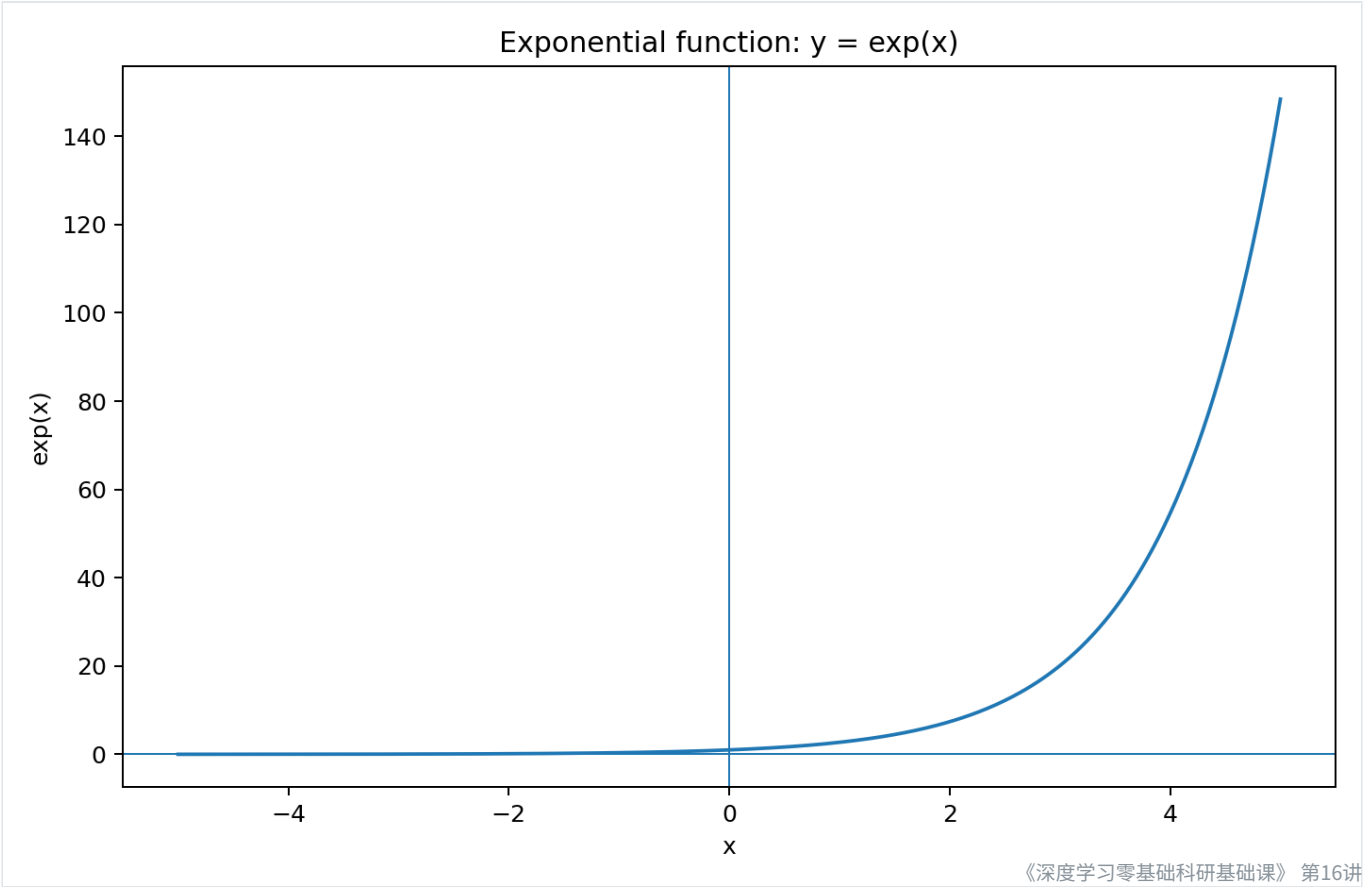
自然指数函数写作：

$$\exp(x) = e^x \quad \text{其中 } e \approx 2.71828$$

你不需要研究 e 的完整数学来源。深度学习阶段只要抓住三个性质：

- 1. **始终为正**：不论 x 是正、负还是 0， e^x 都大于 0。
- 2. **保持顺序**：如果 $a>b$ ，那么 $e^a>e^b$ 。
- 3. **会放大差异**：分数相差 1，指数值的比值就是 $e\approx2.718$ 。

x	e^x
-2	0.1353
-1	0.3679
0	1
1	2.7183
2	7.3891



4. 为什么 exp 很适合处理 logits：正数 + 相对差异

假设两个 logits 分别是 a 和 b，那么：

$$\exp(a) / \exp(b) = \exp(a-b)$$

这意味着相对比值只由两者的差决定。

数字例子： a=2, b=1。则 $\exp(2)=7.389$, $\exp(1)=2.718$ ，两者比值 $7.389/2.718 \approx 2.718$ ，也就是 $\exp(1)$ 。

另一个重要性质是：给所有 logits 同时加上同一个常数 c：

$$\exp(a+c) / \exp(b+c) = \exp(a-b)$$

所以相对关系不变。下一讲你会看到，这正是 Softmax 可以“整体平移不变”的原因。

科研直觉： exp 不是随便选的“把负数变正”工具。它既保证正数，又把分数差异转化为稳定的相对比值结构，这和概率归一化非常契合。

《深度学习零基础科研基础课》第16讲

5. 对数 ln(x)：把 exp 倒过来理解

自然对数写作 $\ln(x)$ 或 $\log(x)$ 。在深度学习文献里，若没有特别说明，很多公式中的 $\log$ 默认指自然对数。

$$y = \ln(x) \iff x = \exp(y)$$

因此：

$$\ln(\exp(x)) = x \quad \exp(\ln(x)) = x \quad (x > 0)$$

x	ln(x)
0.5	-0.6931
1	0
2	0.6931
5	1.6094

定义域：实数范围内 $\ln(x)$ 只能接收 $x > 0$ 。 $\ln(0)$ 不存在， $\ln(\text{负数})$ 也不是普通实数。这一点以后解释为什么代码里要避免直接 $\log(0)$ 很重要。

6. 对数最重要的性质：乘法可以变成加法

$$\ln(a \times b) = \ln(a) + \ln(b)$$

这一条对 LLM 极其重要。语言模型生成一个序列时，整个序列的概率可以写成多个条件概率的乘积：

$$P(x_1, x_2, \dots, x_T) = \prod P(x_t | x_{<t})$$

取 log 后：

$$\ln P(\text{sequence}) = \sum \ln P(x_t | x_{<t})$$

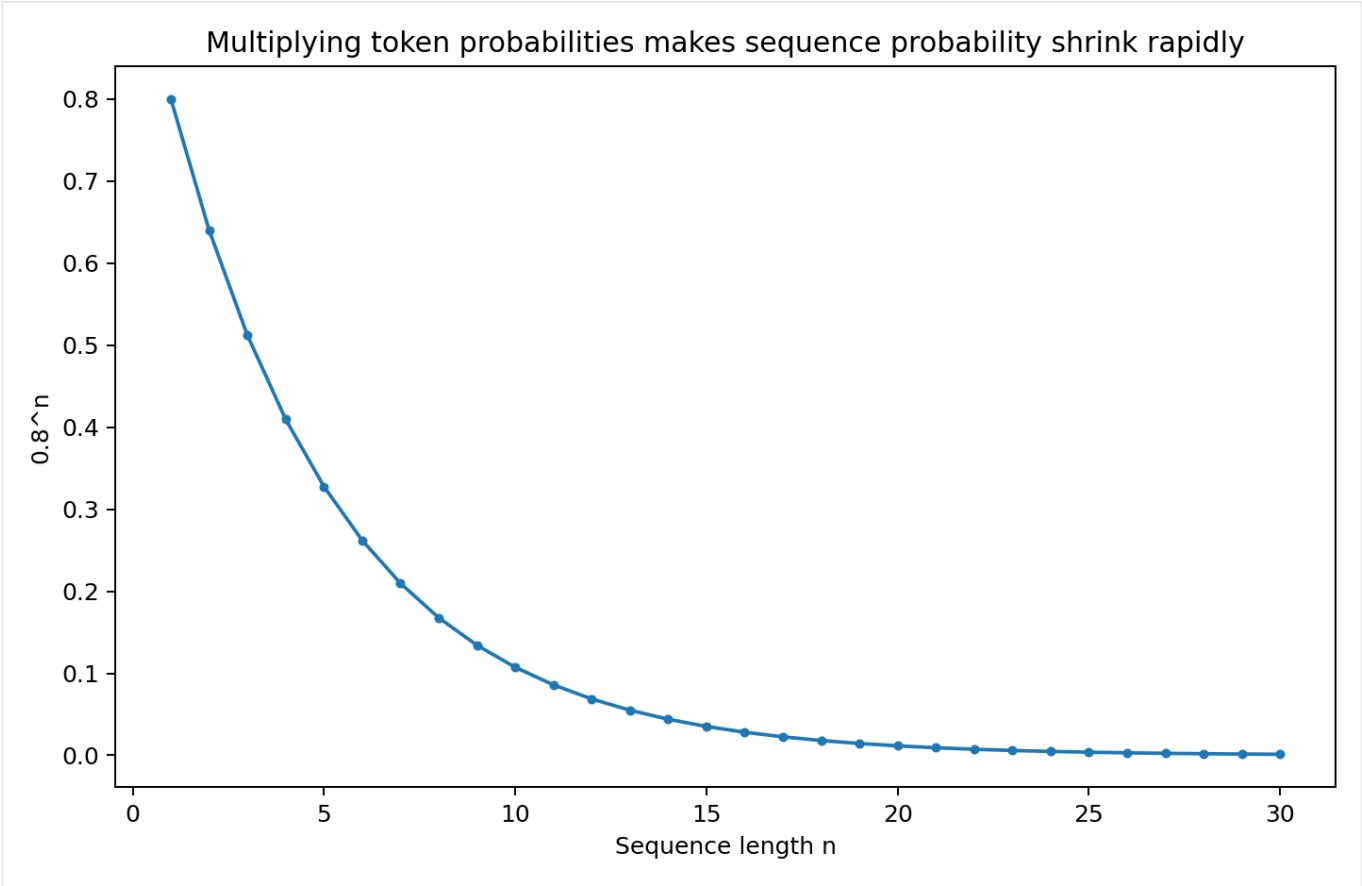
数字例子：三个 token 的条件概率是 [0.8, 0.7, 0.9]。序列概率 = $0.8 \times 0.7 \times 0.9 = 0.504$ ；而 $\ln(0.504) = \ln(0.8) + \ln(0.7) + \ln(0.9) \approx -0.6852$ 。

为什么有用：很多小于 1 的概率不断相乘会快速变得极小；log 把连乘变成求和，使数学分析和数值计算都更容易。

7. 为什么序列越长，概率乘积会迅速变小

假设模型每一步都给正确 token 0.8 的概率。单步看起来并不低，但长度为 n 的序列概率是：

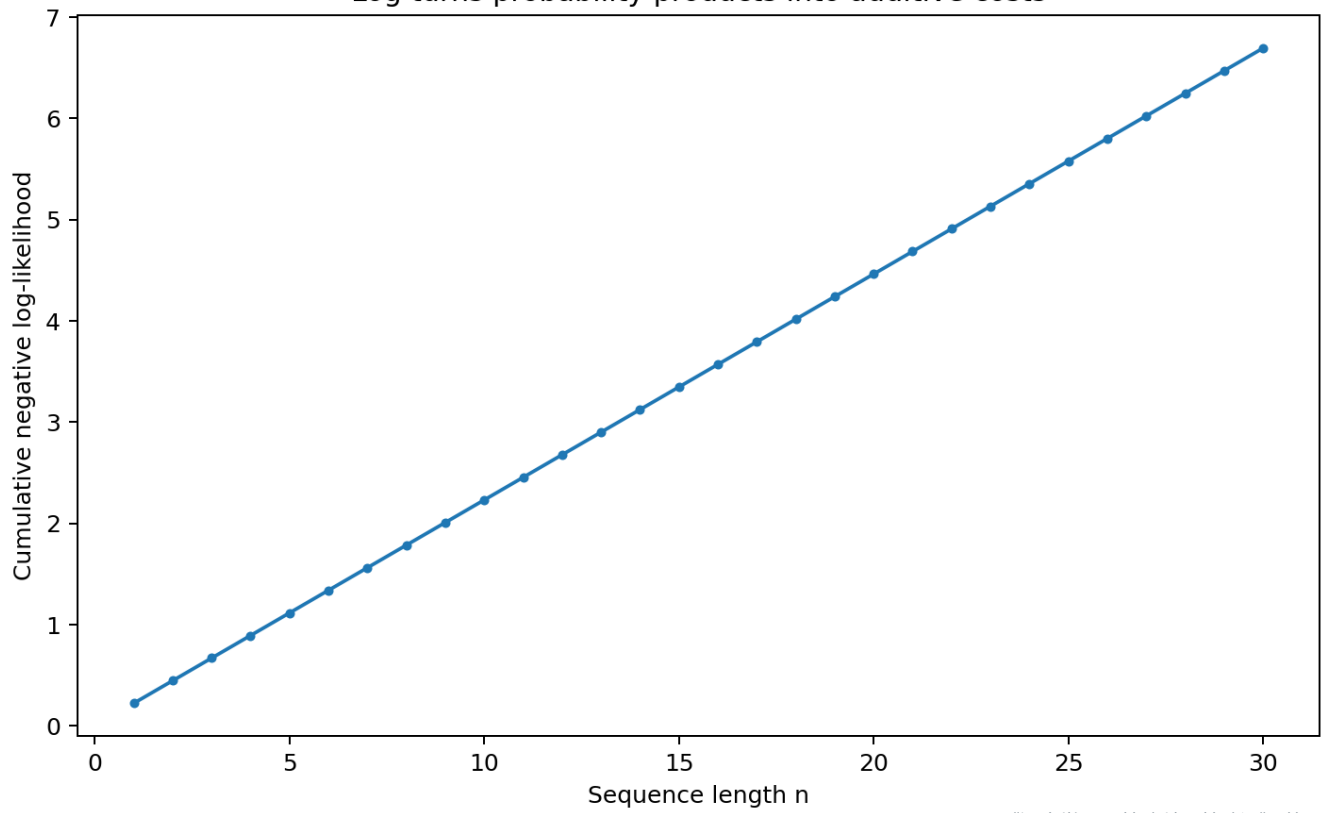
$$P(\text{sequence}) = 0.8^n$$



n	0.8^n	累计 -log 概率
1	0.800000	0.2231
10	0.107374	2.2314
20	0.011529	4.4629
30	0.001238	6.6943

概率乘积快速趋近 0，而负对数代价则以加法方式平稳累积：

Log turns probability products into additive costs



8. $-\log(p)$: 为什么低概率会变成大惩罚

对一个正确答案，如果模型给它概率 p ，我们希望：

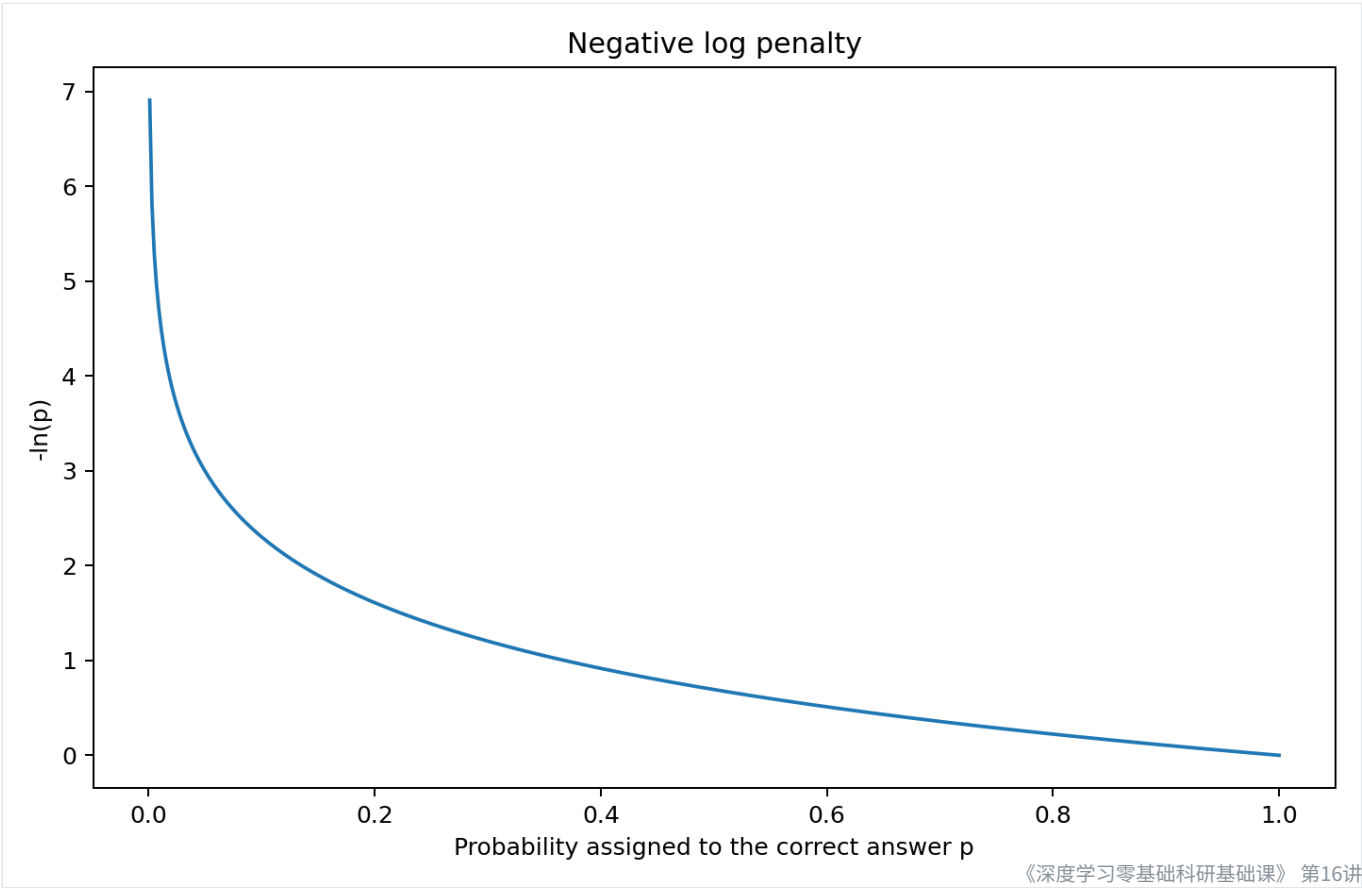
- p 越接近 1，惩罚越小；
- p 越接近 0，惩罚越大。

函数：

Loss = $-\ln(p)$

恰好满足这个要求。

正确答案概率 p	$-\ln(p)$	直觉
0.90	0.1054	很有把握且正确，惩罚小
0.50	0.6931	一般把握
0.10	2.3026	正确答案只给 10%，惩罚大
0.01	4.6052	几乎排除正确答案，惩罚非常大



9. 从 raw score 到概率：先预览 Softmax 的两个动作

如果 logits=[-2,1,0.1]，直接“除以总和”会得到：

$$[-2,1,0.1] / (-0.9) = [2.222,-1.111,-0.111]$$

这显然不是概率，因为出现负数和大于 1 的值。

先做 exp：

$$\exp([-2,1,0.1]) \approx [0.1353,2.7183,1.1052]$$

所有值变成正数；再除以总和，就得到：

$$[0.0342,0.6867,0.2791] \quad \text{且总和}=1$$

下一讲：“exp + 除以 exp 总和”就是 Softmax 的核心。但本讲先理解这两个数学积木，不急着背 Softmax 公式。

10. 数值稳定性：为什么 exp 不能只会照公式写

指数增长非常快。例如 logits=[1000,1001,999]，直接计算 $\exp(1000)$ 在普通浮点数中会溢出为 inf，后面 inf/inf 会产生 nan。

问题：数学公式正确，不代表直接按公式写代码就数值稳定。

常用办法是先减去最大值：

$$z_i' = z_i - \max(z)$$

于是：

$$[1000, 1001, 999] \rightarrow [-1, 0, -2]$$

再做 exp 就是 [0.3679, 1, 0.1353]，既不会溢出，也不改变最终归一化后的相对概率。

科研/工程判断：以后看到论文公式和代码实现略有不同，不要马上认为“代码改了算法”。很多差异只是为了数值稳定，例如 log-sum-exp trick、减最大值、加 epsilon。

《深度学习零基础科研基础课》第16讲

11. Shape：在 LLM 里这些运算发生在哪个维度

对一个普通分类 Batch：

```
logits.shape = (B, C)
```

每个样本都有 C 个类别分数，最终要在最后一个维度 C 上形成概率分布。

对 LLM：

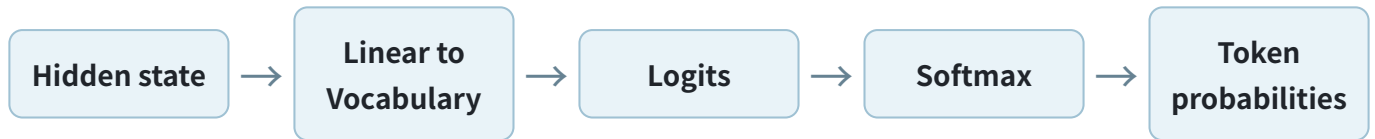
```
logits.shape = (B, T, V)
```

维度	含义
B	Batch 中有多少条序列
T	每条序列有多少 Token 位置
V	Vocabulary 中有多少候选 Token

因此每个 (batch, token_position) 都对应一个长度为 V 的 raw score 向量，Softmax 通常沿最后的 V 维计算。

Shape 直觉：exp 是逐元素运算，因此不改变 Shape；归一化时会沿某个轴求和，但如果保留该轴用于广播，最终 probability Tensor 的 Shape 仍与 logits 一样。

12. 与 LLM / Transformer / Agent Systems 的关系



你以前在 LLM 课程中已经见过 “Vocabulary logits $\rightarrow$ Softmax $\rightarrow$ Next-token probability”。现在我们把其中两个数学原因补上了：

- **exp**: 把任意 logits 变成正数，并保留/放大相对分数关系。
- **log**: 把正确 token 的低概率变成大惩罚，并把长序列概率的乘法变成加法。

Agent Systems 里多数时候不会亲自训练基础模型，但你读模型训练、微调、RL、policy likelihood、token logprob、confidence、sampling 代码时都会反复遇到 probability / logprob。

科研最低能力：以后看到 $\text{logprob} = -2.3$ 时，要知道它不是“概率为 -2.3”。真正概率约为 $\exp(-2.3) \approx 0.10$ 。

《深度学习零基础科研基础课》第16讲

13. 常见误区：这些地方以后最容易读错论文或代码

误区	正确理解
logit=5 就是 50% 或 5%	logit 是无约束 raw score，不是概率。
概率最大的类别一定“可靠”	只代表相对最大；模型可能整体过度自信或校准不良。
logprob 是负概率	logprob 是概率的对数； $0 < p \leq 1$ 时 $\ln(p) \leq 0$ 。
$-\log(p)$ 只是人为技巧	它与最大似然、信息量、Cross Entropy 都有深层联系；当前先掌握计算直觉。
exp 只负责“把负数变正”	还保留顺序，并把分数差转成指数比值。
按数学公式直接写代码总没问题	exp/log 很容易出现 overflow、underflow、log(0)，必须考虑数值稳定性。

14. 本讲总结：为 Softmax / Cross Entropy 搭好最后一块地基

Probability: $0 \leq p \leq 1$, $\sum p = 1$

$\exp(x) > 0$ 且 $\exp(a)/\exp(b) = \exp(a-b)$

$\ln(\exp(x)) = x$ 且 $\ln(ab) = \ln(a) + \ln(b)$

正确答案惩罚： $-\ln(p)$

- 概率告诉我们“候选答案的相对可能性”。
- 指数把任意实数分数变成正数，并自然表达相对差异。
- 对数是指数的逆运算，可以把乘法变成加法。
- $-\log(p)$ 让低概率正确答案得到大惩罚。
- LLM 的 token probability、logprob、sequence likelihood 都建立在这些基础上。

下一讲：正式进入第17讲 Softmax。我们会从 logits=[2,1,0.1] 开始，逐项计算 exp、求和、归一化，并解释为什么输出总和为1、为什么减最大值不改变结果。

《深度学习零基础科研基础课》第16讲